SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: COMPUTER NETWORKS
COURSE CODE: CSA07

COURSE OUTCOME COVERED

CO5: Measure network performance using key metrics with simulation tools. *(BL4,BL5)*

Assessment Tool Weightage: 30%

Annexure A

Pseudocode Writing Task (Algorithm Design)

Code of Conduct:

I M.HARINI(192424011) *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
M.HARINI

Annexure A

Pseudocode Writing Task (Algorithm Design)

1. In a network simulation using Cisco Packet Tracer, a student records total data received as 8000 bits in 4 seconds and observes that 200 packets were sent while 180 packets were received. Write a pseudocode algorithm to calculate the network throughput in bits per second and the packet loss percentage based on the given values. The algorithm should include validation to ensure that time and total packets sent are not zero to avoid errors. It should display both the throughput and packet loss results clearly. Add logic to classify the network as Efficient if throughput is greater than 1500 bps and packet loss is less than 10%, otherwise classify it as Inefficient. Finally, ensure the algorithm outputs a complete summary of the network performance based on the calculated metrics.
2. An organization has multiple network links connecting its branches. The administrator wants to monitor bandwidth utilization and identify overloaded links. Write a pseudocode algorithm that periodically collects the bandwidth usage of each network link and stores the values in an array or list. Use a loop to continuously monitor all links. Add conditions to detect when bandwidth utilization exceeds 80% and recommend switching traffic to a backup link. Explain how this algorithm supports efficient bandwidth management and prevents network congestion.
3. A company has six branch offices connected through multiple network links. Each link has different bandwidth, delay, and utilization values, which change throughout the day. The network administrator wants to continuously identify the best communication path between the headquarters and each branch based on overall link quality rather than just the shortest distance.Write a pseudocode algorithm that periodically collects the bandwidth, delay, and utilization of every network link and

stores the values in suitable arrays or lists. Use a loop to repeat the monitoring process at regular intervals. Calculate a link quality score for each path using the collected metrics and select the path with the highest score. Include conditions to detect when a link becomes overloaded or fails, automatically selecting an alternate path and generating an alert for the administrator. Explain how your algorithm improves network reliability, routing efficiency, and fault tolerance while adapting to changing network conditions.

RUBRICS FOR CALCULATION

Criteria	**Weightage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized	Handles most cases; reasonable	Handles basic cases only	Incomplete or inefficient
		approach	efficiency		

1. Network Throughput and Packet Loss

Pseudocode

```
BEGIN
   totalDataReceived ← 8000
   time ← 4
   packetsSent ← 200
   packetsReceived ← 180
   IF time = 0 THEN
      DISPLAY "Error: Time cannot be zero."
      STOP
   END IF
   IF packetsSent = 0 THEN
      DISPLAY "Error: Total packets sent cannot be zero."
      STOP
   END IF
   throughput ← totalDataReceived / time
   packetsLost ← packetsSent - packetsReceived
   packetLossPercentage ← (packetsLost / packetsSent) × 100
   DISPLAY "----- Network Performance Summary -----"
   DISPLAY "Total Data Received: ", totalDataReceived, " bits"
   DISPLAY "Time: ", time, " seconds"
   DISPLAY "Packets Sent: ", packetsSent
   DISPLAY "Packets Received: ", packetsReceived
   DISPLAY "Packets Lost: ", packetsLost
   DISPLAY "Throughput: ", throughput, " bps"
   DISPLAY "Packet Loss: ", packetLossPercentage, "%"
   IF throughput > 1500 AND packetLossPercentage < 10 THEN
      networkStatus ← "Efficient"
   ELSE
      networkStatus ← "Inefficient"
   END IF
   DISPLAY "Network Classification: ", networkStatus
   IF networkStatus = "Efficient" THEN
      DISPLAY "Summary: The network has good throughput and acceptable packet loss."
   ELSE
      DISPLAY "Summary: The network performance needs improvement."
   END IF
END
```

Expected Output

- **Throughput** = 8000 / 4 = **2000 bps**
- **Packets Lost** = 200 − 180 = **20 packets**
- **Packet Loss** = (20 / 200) × 100 = **10%**
- **Classification** = **Inefficient**

The network is classified as inefficient because, although the throughput is above 1500 bps, packet loss is **10%**, which is not less than 10%.

2. Bandwidth Utilization and Overloaded Link Detection

Pseudocode

```
BEGIN
   links ← ["Link1", "Link2", "Link3", "Link4"]
   utilization[4]
   monitoringInterval ← 10    // seconds
   WHILE TRUE DO
      DISPLAY "----- Bandwidth Monitoring -----"
      FOR i ← 0 TO LENGTH(links) - 1 DO
         utilization[i] ← GET_BANDWIDTH_UTILIZATION(links[i])
         DISPLAY links[i], " Utilization: ", utilization[i], "%"
         IF utilization[i] > 80 THEN
            DISPLAY "WARNING: ", links[i], " is overloaded."
            DISPLAY "Recommendation: Switch traffic to backup link."
            backupLink ← FIND_AVAILABLE_BACKUP_LINK(links[i])
            IF backupLink IS AVAILABLE THEN
               SWITCH_TRAFFIC(links[i], backupLink)
               DISPLAY "Traffic switched to ", backupLink
            ELSE
               DISPLAY "No backup link available."
            END IF
         ELSE
            DISPLAY links[i], " is operating within acceptable limits."
         END IF
      END FOR
      WAIT monitoringInterval seconds
   END WHILE
END
```

Explanation

The algorithm continuously monitors every network link using a loop. The current bandwidth utilization of each link is collected and stored in the utilization array. When utilization exceeds **80%**, the algorithm identifies the link as overloaded and recommends moving traffic to a backup link. If a backup link is available, traffic is automatically switched to it.
This approach helps:

- **Monitor bandwidth continuously** rather than relying on occasional checks.
- **Detect congestion early** when utilization exceeds 80%.
- **Balance network traffic** by moving traffic away from overloaded links.
- **Reduce network congestion** and improve application performance.
- **Improve reliability** by using backup links when required.
- **Support efficient bandwidth management** through continuous measurement and automatic response.

3. Dynamic Network Path Selection Based on Link Quality

Pseudocode

```
BEGIN
   branches ← ["Branch1", "Branch2", "Branch3",
            "Branch4", "Branch5", "Branch6"]
   bandwidth[]
   delay[]
   utilization[]
   linkStatus[]
   qualityScore[]
   monitoringInterval ← 10    // seconds
   WHILE TRUE DO
      DISPLAY "===== Network Path Monitoring ====="
      FOR each link IN networkLinks DO
         bandwidth[link] ← GET_BANDWIDTH(link)
         delay[link] ← GET_DELAY(link)
         utilization[link] ← GET_UTILIZATION(link)
         linkStatus[link] ← GET_STATUS(link)
         IF linkStatus[link] = "FAILED" THEN
            qualityScore[link] ← 0
            DISPLAY "ALERT: ", link, " has failed."
         ELSE
            IF utilization[link] > 80 THEN
               DISPLAY "WARNING: ", link, " is overloaded."
            END IF
            qualityScore[link] ←
               (NORMALIZE(bandwidth[link]) × 0.50)
               + ((1 - NORMALIZE(delay[link])) × 0.25)
               + ((1 - NORMALIZE(utilization[link])) × 0.25)
         END IF
      END FOR
      FOR each branch IN branches DO
         possiblePaths ← FIND_ALL_AVAILABLE_PATHS(
                     Headquarters, branch)
         bestPath ← NULL
         highestScore ← -1
         FOR each path IN possiblePaths DO
            pathAvailable ← TRUE
            pathScore ← 0
            FOR each link IN path DO
               IF linkStatus[link] = "FAILED" THEN
                  pathAvailable ← FALSE
                  BREAK
               END IF
               IF utilization[link] > 80 THEN
                  pathAvailable ← FALSE
                  BREAK
               END IF
               pathScore ← pathScore + qualityScore[link]
            END FOR
            IF pathAvailable = TRUE THE
               averagePathScore ←
                  pathScore / LENGTH(path)
```

```
                IF averagePathScore > highestScore THEN
                highestScore ← averagePathScore
                bestPath ← path
              END IF
            END IF
          END FOR
          IF bestPath IS NOT NULL THEN
            SET_ROUTE(Headquarters, branch, bestPath)
            DISPLAY "Branch: ", branch
            DISPLAY "Selected Best Path: ", bestPath
            DISPLAY "Path Quality Score: ", highestScore
          ELSE
            DISPLAY "ALERT: No suitable path available for ", branch
          END IF
        END FOR
        WAIT monitoringInterval seconds
    END WHILE
END
```

Explanation

This algorithm uses **bandwidth, delay, and utilization** instead of simply selecting the shortest physical route. Each link receives a quality score based on these metrics.
A possible scoring model is:
Link Quality Score =
(Bandwidth × 50%)
+ (Low Delay × 25%)
+ (Low Utilization × 25%)
The values are normalized before calculating the score so that bandwidth, delay, and utilization can be compared fairly.
The algorithm continuously:

1. Collects bandwidth, delay, utilization, and link-status information.
2. Stores the measurements in arrays/lists.
3. Calculates a quality score for each available link.
4. Finds possible paths from headquarters to each of the six branches.
5. Calculates the overall score of each path.
6. Selects the path with the **highest quality score**.
7. Detects failed or overloaded links.
8. Removes unsuitable links from path selection.
9. Automatically selects an alternate path when necessary.
10. Generates alerts when links fail or become overloaded.

Benefits

Reliability: Failed links are detected and removed from active path selection.
Routing efficiency: The algorithm chooses paths based on actual network conditions rather than only distance.
Fault tolerance: Alternate paths can automatically be selected when the preferred path fails or becomes overloaded.
Adaptive performance: Because bandwidth, delay, and utilization are periodically updated, routing decisions change as network conditions change.
Reduced congestion: Links with utilization above 80% can be avoided, helping distribute traffic across healthier paths.
Scalability: The same approach can be extended to additional branches and network links by adding them to the corresponding arrays/lists.